Arquitetura do Sistema — RAG Assistente PT

Visão Geral

Assistente conversacional em português para documentação de bibliotecas de análise de dados (pandas, numpy, matplotlib, seaborn). Arquitetura full-stack com pipeline RAG (Retrieval-Augmented Generation), frontend React e backend FastAPI deployado em nuvem.

Stack

Camada	Tecnologia
Frontend	Next.js 13 (App Router) + Tailwind CSS
Backend	FastAPI + Uvicorn (Python 3.12)
Vector Store	ChromaDB (persistente)
Embedder	`intfloat/multilingual-e5-small` (384 dims)
Reranker	`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`
LLM Principal	Groq — `llama-3.1-8b-instant`
LLM Fallback	Cerebras — `llama-3.3-70b`
Tradução	Google Gemini API (`gemini-2.0-flash`) → fallback Groq
Deploy Backend	Hugging Face Spaces (Docker)
Deploy Frontend	Vercel

Diagrama de Fluxo



```
│  FastAPI  |  porta 7860                  │
│                                          │
│  POST /ask                               │
│    └▶ rag_query.retrieve()               │
│         └▶ ChromaDB (top-10)             │
│    └▶ rag_query.rerank()                 │
│         └▶ CrossEncoder (top-3)          │
│    └▶ build_pt_prompt()                  │
│    └▶ Groq API ──(fallback)──▶ Cerebras  │
│    └▶ format_citations()                 │
│                                          │
│  GET /health                             │
│    └▶ models_loaded + chroma_count       │
└──────────────────────────────────────────┘
```

Estrutura de Diretórios

```
micro_saas_ods_2/
├── app/                        # Frontend Next.js
│   ├── app/
│   │   ├── page.tsx            # Interface principal do chat
│   │   ├── layout.tsx          # Root layout
│   │   └── globals.css
│   ├── components/ui/          # Badge, Button, Card, Input, Collapsible
│   ├── hooks/
│   │   └── useChatHistory.ts   # Estado de chat via localStorage
│   ├── next.config.js
│   └── vercel.json
│
├── backend/
│   ├── app.py                  # Entry point FastAPI (lifespan + rotas)
│   └── requirements.txt
│
├── src/                        # Núcleo do pipeline RAG
│   ├── config.py               # Configuração centralizada
│   ├── rag_query.py            # Retrieve → Rerank → LLM → Citations
│   ├── indexer.py              # Embed + upsert no ChromaDB
│   ├── html_extractor.py       # Extração de docs oficiais (trafilatura)
│   ├── medium_extractor.py     # Extração de artigos Medium
│   ├── translator.py           # Tradução EN→PT com preservação de código
│   ├── glossary_repair.py      # Correção de termos técnicos mal traduzidos
│   ├── baseline_ingest.py      # Pipeline baseline (EN)
│   └── eval/
│       ├── ragas_evaluator.py  # Métricas RAGAS (sem LLM)
│       └── ragas_dashboard.py  # Visualização dos resultados
│
├── data/
│   ├── main.py                 # Pipeline de ingestão de dados
│   ├── chroma_db/              # Índice vetorial persistente
│   ├── {lib}/pt/               # Docs traduzidos (HTML)
```

```
│   ├── {lib}/en/               # Docs originais (HTML)
│   ├── medium/                 # Artigos Medium
│   ├── eval/                   # Datasets e resultados de avaliação
│   └── chunks.jsonl            # Chunks serializados
│
├── scripts/
│   ├── smoke_test.py           # Smoke test do backend
│   └── deploy_space.py         # Deploy para HF Space
│
├── tests/                      # Pytest (unit + integration)
├── docs/                       # Documentação do projeto
├── Dockerfile                  # Imagem para HF Space
├── pyproject.toml              # Deps + metadados (uv)
└── .env                        # Chaves de API
```

Pipeline RAG

Ingestão (offline)

```
Docs HTML (EN)
    │
    ├─▶ html_extractor.py (trafilatura → Markdown)
    │
    ├─▶ translator.py
    │     ├─ Extrai code blocks
    │     ├─ Traduz texto: Gemini → fallback Groq
    │     ├─ Restaura code blocks
    │     └─ glossary_repair.py (regex determinístico)
    │
    ├─▶ Chunker (450 tokens max, 80 words overlap)
    │
    └─▶ indexer.py
          ├─ Embed: e5-small ("passage: " prefix)
          └─ Upsert: ChromaDB (collection: rag_chunks_pt)
```

Consulta (online — por request)

```
Pergunta do usuário (PT)
    │
    ├─▶ Encode: e5-small ("query: " prefix)
    │
    ├─▶ ChromaDB similarity search → top-10 chunks
    │
    ├─▶ CrossEncoder rerank → top-3 chunks
    │
    ├─▶ build_pt_prompt()
    │     ├─ System: role PT-BR + instrução de fallback
    │     └─ Context: [1]...[3] com metadados
```

```
      │
      ├─▶ Groq (llama-3.1-8b-instant)
      │     └─ fallback: Cerebras (llama-3.3-70b)
      │
      └─▶ JSON: { answer, citations: [{library, section, source}] }
```

Coleções ChromaDB

Coleção	Pipeline	Embedder	Idioma	Uso
`rag_chunks_pt`	Novo (principal)	e5-small	PT-BR	Produção
`rag_chunks_baseline_en`	Baseline	MiniLM-L6-v2	EN	A/B testing

Metadados por chunk: `library`, `section`, `source`, `language`, `source_type`

API — Endpoints

`POST /ask`

```
// Request
{ "question": "Como usar groupby no pandas?", "top_n": 5 }

// Response
{
  "answer": "O groupby no pandas...",
  "citations": [
    { "library": "pandas", "section": "GroupBy", "source": "..." }
  ]
}
```

`GET /health`

```
{
  "status": "ok",
  "models_loaded": true,
  "chroma_count": 4823
}
```

CORS: habilitado para todas as origens (`allow_origins=["*"]`)

Frontend

Funcionalidade	Implementação
Histórico de chat	`useChatHistory` — localStorage
Warm-up cold start	`GET /health` no mount
Code blocks	Syntax highlighting + botão de cópia
Citations	Chips filtráveis por biblioteca
Textarea	Auto-resize, Shift+Enter para quebra de linha
Sidebar	Renomear, deletar, timestamps relativos
Quick prompts	Perguntas pré-definidas por biblioteca

Variáveis de Ambiente

```
# LLM Principal
GROQ_API_KEY=gsk_...
GROQ_LLM_MODEL=llama-3.1-8b-instant

# LLM Fallback
CEREBRAS_API_KEY=csk_...
CEREBRAS_LLM_MODEL=llama3.1-8b

# Tradução
GOOGLE_API_KEY=...
GEMINI_TRANSLATION_MODEL=gemini-2.0-flash

# HF Hub (download de modelos)
HF_TOKEN=hf_...

# Frontend (Vercel)
NEXT_PUBLIC_BACKEND_URL=https://<space>.hf.space
```

Hiperparâmetros

Parâmetro	Valor	Motivo
`TOP_K_RETRIEVE`	10	Equilíbrio entre cobertura e velocidade de reranking
`TOP_K_RERANK`	3	~31% menos tokens em relação ao top-5
`CHUNK_MAX_TOKENS`	450	Granularidade sem exceder o contexto do LLM
`CHUNK_OVERLAP_WORDS`	80	Continuidade semântica entre chunks
`LLM_MAX_TOKENS`	500	Respostas concisas (reduzido de 800)
`LLM_TEMPERATURE`	0.2	Respostas determinísticas e factuais

Avaliação (RAGAS)

Métricas calculadas localmente, sem chamadas LLM:

Métrica	Descrição
`answer_semantic_similarity`	Similaridade cosseno entre resposta e ground truth
`context_recall`	% de tokens do ground truth presente nos contextos
`context_precision`	Similaridade semântica média entre contextos e pergunta

Artefatos: `ragas_results_optimized.json`, `ragas_report.html`

Deploy

Backend — HF Space

```
Dockerfile → HF Space (porta 7860)
├── Python 3.12-slim
├── Torch CPU-only
├── ChromaDB pré-populado (data/chroma_db/)
└── Modelos lazy-loaded no lifespan do FastAPI
```

O índice ChromaDB é gerado localmente e enviado via `scripts/deploy_space.py`.

Frontend — Vercel

```
app/ → Vercel
└── NEXT_PUBLIC_BACKEND_URL aponta para o HF Space
```

Duas Variantes de Pipeline

O sistema suporta dois pipelines alternáveis via parâmetro `variant`:

	Pipeline PT (padrão)	Pipeline EN (baseline)
Coleção	`rag_chunks_pt`	`rag_chunks_baseline_en`
Embedder	e5-small	MiniLM-L6-v2
Reranker	mmarco-mMiniLMv2	ms-marco-MiniLM-L-6-v2
Prompt	PT-BR	EN
Idioma docs	PT	EN